

SOLID

07 May 2026 10:56 PM

In a large system we face many problems mostly are related to code maintainability, readability etc. To overcome it SOLID principles are introduced.

S: Single Responsibility Principle
O: Open close Principle
L: Liskov Substitution Principle
I: Interface Segregation Principle
D: Dependency Injection Principle

S(Single Responsibility Principle) : Single class should have only one responsibility. It should do one thing.

Eg: A Product class contains properties such as Name and Price, but it also includes methods like ShoppingCart() and GetInvoice(). This design violates the **Single Responsibility Principle (SRP)** because the class is handling multiple responsibilities.

A Product class should only be responsible for representing product-related data and behavior. Managing shopping cart operations and generating invoices are separate responsibilities and should be handled by their own classes, such as ShoppingCart and InvoiceService.

If all these responsibilities are placed in the Product class, the code becomes tightly coupled. Any change to cart logic or invoice generation would require modifying the Product class, increasing the risk of introducing bugs and making the code harder to maintain.

To follow SRP, each responsibility should be moved to its own dedicated class:

- Product → stores product information (Name, Price)
- ShoppingCart → manages adding/removing products
- InvoiceService → generates invoices

```
using System;
using System.Collections.Generic;
public class Product
{
    public string Name { get; set; }
    public decimal Price { get; set; }
    public Product(string name, decimal price)
    {
        Name = name;
        Price = price;
    }
}
public class Cart
{
    public List<Product> products=new List<Product>();

    public void AddProduct(Product product)
    {
        products.Add(product);
        Console.WriteLine($"Added {product.Name} to cart.");
    }
    public decimal CalculateTotal()
    {
        decimal total = 0;
        foreach (var product in products)
        {
            total += product.Price;
        }
        return total;
    }
}
public class CartPrinter
{
    private Cart cart;
    public CartPrinter(Cart cart)
    {
        this.cart = cart;
    }
    public void PrintCart()
    {
        Console.WriteLine("Cart Contents:");
        foreach (var product in cart.products)
        {
            Console.WriteLine($"{product.Name} - ${product.Price}");
        }
        Console.WriteLine($"Total: ${cart.CalculateTotal()}");
    }
}

public class SingleResponsibility
{
    public static void Main(string[] args)
    {
    }
```

```

        Cart cart = new Cart();
        cart.AddProduct(new Product("Laptop", 999.99m));
        cart.AddProduct(new Product("Mouse", 49.99m));
        CartPrinter printer = new CartPrinter(cart);
        printer.PrintCart();
    }
}

```

Open close Principle: A class should be open for extension and closed for modification. If we have any add on feature do not change the old code.

To achieve the Open/Closed Principle, object-oriented programming concepts such as:

- **Abstraction**
- **Inheritance**
- **Polymorphism**

are commonly used.

For example, a base abstract class or interface can define common behaviour, and new classes can inherit from it and provide their own implementations. This allows the system to support new requirements without modifying the existing codebase.

```

public class Product
{
    public string Name { get; set; }
    public decimal Price { get; set; }
    public Product(string name, decimal price)
    {
        Name = name;
        Price = price;
    }
}

public class Cart
{
    public List<Product> products=new List<Product>();

    public void AddProduct(Product product)
    {
        products.Add(product);
        Console.WriteLine($"Added {product.Name} to cart.");
    }
    public decimal CalculateTotal()
    {
        decimal total = 0;
        foreach (var product in products)
        {
            total += product.Price;
        }
        return total;
    }
}

public class CartPrinter
{
    private Cart cart;
    public CartPrinter(Cart cart)
    {
        this.cart = cart;
    }
    public void PrintCart()
    {
        Console.WriteLine("Cart Contents:");
        foreach (var product in cart.products)
        {
            Console.WriteLine($"{product.Name} - ${product.Price}");
        }
        Console.WriteLine($"Total: ${cart.CalculateTotal()}");
    }
}

interface Persistence
{
    void Save(Cart cart);
}

public class FilePersistence : Persistence
{
    public void Save(Cart cart)
    {
        // Simulate saving to a file
        Console.WriteLine("Cart saved to file.");
    }
}

public class DatabasePersistence : Persistence
{
    public void Save(Cart cart)
    {
        // Simulate saving to a database
        Console.WriteLine("Cart saved to database.");
    }
}

```

```

}
class OCP
{
    public static void Main(String[] args)
    {
        Cart cart = new Cart();
        cart.AddProduct(new Product("Laptop", 999.99m));
        cart.AddProduct(new Product("Mouse", 49.99m));
        CartPrinter printer = new CartPrinter(cart);
        printer.PrintCart();
        Persistence filePersistence = new FilePersistence();
        filePersistence.Save(cart);
        Persistence dbPersistence = new DatabasePersistence();
        dbPersistence.Save(cart);
    }
}

```

Liskov Substitution Principle: Sub-classes should be substitutable for their base classes.

If class B is a subclass of class A, then an object of class B should be able to replace an object of class A without causing any errors or changing the expected behavior of the program.

Suppose we have an interface Account with two methods:

- Deposit()
- Withdraw()

And the following classes implement this interface:

- SavingsAccount
- CurrentAccount
- FixedDepositAccount

A FixedDepositAccount generally does not allow withdrawals before maturity. If its Withdraw() method throws an exception or is left unsupported, then it cannot behave as a proper substitute for Account.

```

interface DepositOnlyAccount
{
    void Deposit(decimal amount);
}
interface WithdrawOnlyAccount
{
    void Withdraw(decimal amount);
}
public class SavingsAccount : WithdrawOnlyAccount
{
    private decimal balance;
    public SavingsAccount(decimal initialBalance)
    {
        balance = initialBalance;
    }
    public void Withdraw(decimal amount)
    {
        if (amount > balance)
        {
            Console.WriteLine("Insufficient funds!");
            return;
        }
        balance -= amount;
        Console.WriteLine($"Withdrew {amount}. New balance: {balance}");
    }
}
public class FixedDepositAccount : DepositOnlyAccount
{
    private decimal balance;
    public FixedDepositAccount(decimal initialBalance)
    {
        balance = initialBalance;
    }
    public void Deposit(decimal amount)
    {
        balance += amount;
        Console.WriteLine($"Deposited {amount}. New balance: {balance}");
    }
}
public class LSP
{
    public static void Main(String[] args)
    {
        DepositOnlyAccount depositAccount = new FixedDepositAccount(1000);
        depositAccount.Deposit(500); // Works fine
        WithdrawOnlyAccount withdrawAccount = new SavingsAccount(1000);
        withdrawAccount.Withdraw(200); // Works fine
    }
}

```

Under the **Liskov Substitution Principle (LSP)**, a subclass must be usable anywhere its parent class (or interface) is expected. One important part of this principle is that overridden methods must remain compatible with the parent's method contract.

Method Signature Rule in LSP

When a child class overrides a method, it must preserve the expectations established by the parent class.

This rule is usually discussed in four parts:

1. **Parameter Rule (Preconditions)**
2. **Return Type Rule (Covariance)**
3. **Exception Rule**
4. **Properties Rule**
5. **Behavioral Rules**
 - Invariants Rule
 - History Constraint Rule

1. Parameter Rule

The child should not demand stricter inputs.

Parent

Withdraw(100)

Bad Child

Only allows Withdraw(1000 or more)

The child is more restrictive, which breaks expectations.

2. Return Type Rule

The child can return the same type or a more specific type.

Parent

GetAccount() returns Account

Child

GetAccount() returns SavingsAccount

This is safe because SavingsAccount is an Account.

3. Exception Rule

The child should not suddenly throw unexpected exceptions.

Parent Promise

Deposit() works normally

Bad Child

Deposit() throws NotSupportedException

The child breaks the promise.

4. Properties Rule

The child should preserve the meaning of properties.

Parent Promise

Balance is always a valid account balance

Bad Child

Balance returns -100

This violates expectations.

5. Invariants Rule

Conditions that must always be true should stay true.

Example

Balance >= 0

Every child must keep this rule.

6. History Constraint Rule

The child should not allow invalid changes over time.

Example

A read-only document should remain read-only.

A child should not suddenly add editing behavior.

Interface Segregation Principle (ISP) : A class should not be forced to implement methods that it does not need.

Instead of creating one large interface with many methods, create smaller, focused interfaces so that each class implements only the methods it actually uses.

Bad Eg: Interface Shape{
 public void area();

```

        Public void volume();
    }

```

Eg:

```

interface TwoDShape
{
    public double Area();
}
interface ThreeDShape
{
    public double Volume();
}
public class Circle : TwoDShape
{
    private double r;
    public Circle(double radius)
    {
        r = radius;
    }
    public double Area()
    {
        return Math.PI * r * r;
    }
}
public class Sphere : ThreeDShape
{
    private double r;
    public Sphere(double radius)
    {
        r = radius;
    }
    public double Volume()
    {
        return (4.0 / 3.0) * Math.PI * r * r * r;
    }
}
public class ISP
{
    public static void Main(String[] args)
    {
        TwoDShape circle = new Circle(5);
        Console.WriteLine($"Area of the circle: {circle.Area()}");

        ThreeDShape sphere = new Sphere(5);
        Console.WriteLine($"Volume of the sphere: {sphere.Volume()}");
    }
}

```

Dependency Inversion Principle: High level module should not depend on the low level module but rather both should depend on Abstraction.

Eg:

```

using System;
// Abstraction (Interface)
public interface IDatabase
{
    void Save(string data);
}
// MySQL implementation (Low-Level module)
public class MySQLDatabase : IDatabase
{
    public void Save(string data)
    {
        Console.WriteLine(
            "Executing SQL Query: INSERT INTO users VALUES('" +
            data + "');"
        );
    }
}
// MongoDB implementation (Low-Level module)
public class MongoDBDatabase : IDatabase
{
    public void Save(string data)
    {
        Console.WriteLine(
            "Executing MongoDB Function: db.users.insert({ name: '" +
            data + "' })"
        );
    }
}
// High-Level module (loosely coupled through Dependency Injection)
public class UserService
{
    private readonly IDatabase db;
    // Constructor Injection
}

```

```

public UserService(IDatabase database)
{
    db = database;
}
public void StoreUser(string user)
{
    db.Save(user);
}
}
public class DIPFollowed
{
    public static void Main(string[] args)
    {
        IDatabase mysql = new MySQLDatabase();
        IDatabase mongodb = new MongoDBDatabase();
        UserService service1 = new UserService(mysql);
        service1.StoreUser("Aditya");
        UserService service2 = new UserService(mongodb);
        service2.StoreUser("Rohit");
    }
}

```